

Huawei TechArena 2025: BESS Energy Management System

Repository Status: Currently developing **Round 2 (Phase 2)** solution.
Phase 1 (2024 optimization) artifacts have been archived to `archive_old_files/` .
New Data: `data/TechArena2025_Phase2_data.xlsx`
Active Branch: `r2-with-bat-config`

A Python-based Energy Management System (EMS) for optimizing Battery Energy Storage System (BESS) operations across multiple European electricity markets.

Table of Contents

- [Challenge Context](#)
- [Mathematical Model](#)
- [Investment Optimization](#)
- [Implementation Pipeline](#)
- [Pyomo Framework Overview](#)
- [Usage](#)
- [Key Concepts](#)

Challenge Context

The primary objective is to develop a Python-based Energy Management System (EMS) algorithm for a utility-scale Battery Energy Storage System (BESS). The goal is to optimize the BESS's financial performance by participating in multiple European electricity markets.

Core Optimization Tasks

The challenge is divided into three core, interconnected optimization tasks:

- Operation Optimization:** Maximize the BESS's revenue over the year 2024 by developing an optimal charge/discharge strategy. This involves performing energy arbitrage on the day-ahead wholesale market and placing capacity bids on the Frequency Containment Reserve (FCR) and automatic Frequency Restoration Reserve (aFRR) ancillary service markets.
- Investment Optimization:** Identify which of five European countries (Germany, Austria, Switzerland, Hungary, Czech Republic) offers the highest Return on Investment (ROI) for installing the BESS over a 10-year period. This analysis must incorporate country-specific financial data, such as the Weighted-Average Cost of Capital (WACC) and inflation rates.
- Configuration Optimization:** Determine the optimal BESS configuration by analyzing the impact of different C-rates and daily cycle limits on profitability and performance.

Evaluation Criteria

Submissions are evaluated based on:

- Revenue maximization (50%):** How well the algorithm maximizes revenue based on market prices. A linear scaling formula will be used to map all submission values between the minimum and maximum observed.
- Investment optimization (20%):** Assessment of optimal investment locations and markets.
- Configuration optimization (20%):** Assessment of the most relevant configuration parameters on BESS revenue.
- Code Quality and Documentation (10%):** Clarity and structure of the code.

Market Participation Rules

Feature	Day-Ahead Market (EPEX SPOT)	FCR Capacity Market	aFRR Capacity Market
Mechanism	Blind Auction	Daily Auction	Daily Auction
Gate Closure Time (D-1)	12:00 CET	8:00 CET	9:00 CET

Feature	Day-Ahead Market (EPEX SPOT)	FCR Capacity Market	aFRR Capacity Market
Product Granularity	15 minutes	4 hours	4 hours
Bid Structure	Energy (MWh)	Symmetric Capacity (MW)	Asymmetric Capacity (MW)
Remuneration	Pay-as-Cleared (Energy)	Pay-as-Cleared (Capacity)	Pay-as-Bid (Capacity)
Minimum Bid Size	0.1 MW	1 MW	1 MW

BESS Technical Parameters

Fixed Parameters:

- Nominal Energy Capacity (E_{nom}): 4,472 kWh (Huawei LUNA2000-4.5MWh)
- Rated Power (P_{rated}): 2,236 kW
- Initial Investment Cost (C_{inv}): 200 EUR/kWh
- Charging/Discharging Efficiency: 95%
- SOC Range: 0-100% (full operational range allowed)

Configuration Scenarios:

C-rate	Max Power (kW)	Daily Cycles	Max Daily Discharge (kWh)
0.25 C	1,118	1.0	4,472
0.33 C	1,476	1.5	6,708
0.50 C	2,236	2.0	8,944

Total Scenarios: 6 countries (DE, DE_LU, AT, CH, HU, CZ) × 3 C-rates × 3 cycles = 54 scenarios

Mathematical Model

Objective Function

The objective is to maximize the total net profit over the one-year horizon. This is the sum of day-ahead energy arbitrage revenue and ancillary service capacity payments, minus the cost of energy purchased for charging.

$$maxZ = \sum_{t \in T} [(P_DA(t)/1000 \cdot p_{dis}(t) - P_DA(t)/1000 \cdot p_{ch}(t)) \cdot \Delta t] + \sum_{b \in B} [P_FCR(b) \cdot c_{fcr}(b) + P_aFRR^{pos}(b) \cdot c_{afrr^{pos}}(b) + P_aFRR^{neg}(b) \cdot c_{afrr^{neg}}(b)]$$

Where:

- First term:** Day-ahead net profit (power in kW, price in EUR/MWh)
- Second term:** Ancillary service capacity revenue (bids in MW, prices in EUR/MW)

Constraint Equations

The optimization is subject to the following constraints:

1. State of Charge (SOC) Dynamics

Energy balance of the BESS accounting for charging efficiency, discharging losses, and self-discharge:

$$e_{soc}(t) = e_{soc}(t-1) + [p_{ch}(t) \cdot \eta_{ch} - p_{dis}(t)/\eta_{dis}] \cdot \Delta t \quad \forall t \in T$$

For t=1 (initial interval), use initial SOC:

$$e_{soc}(1) = e_{soc}^{init} + [p_{ch}(1) \cdot \eta_{ch} - p_{dis}(1)/\eta_{dis}] \cdot \Delta t$$

Pyomo Implementation:

```
def soc_dynamics_rule(model, t):
    if t == T_data[0]:
        return model.e_soc[t] == model.E_soc_init + \
            (model.eta_ch * model.p_ch[t] - model.p_dis[t] / model.eta_dis) * model.dt
    else:
        return model.e_soc[t] == model.e_soc[t-1] + \
            (model.eta_ch * model.p_ch[t] - model.p_dis[t] / model.eta_dis) * model.dt

model.soc_dynamics = pyo.Constraint(model.T, rule=soc_dynamics_rule)
```

2. SOC Limits

Enforced through variable bounds to stay within operational boundaries (0-100% allowed per competition rules):

$$SOC_{min} \cdot E_{nom} \leq e_{soc}(t) \leq SOC_{max} \cdot E_{nom} \quad \forall t \in T$$

Pyomo Implementation:

```
model.e_soc = pyo.Var(
    model.T,
    bounds=(self.battery_params['soc_min'] * self.battery_params['capacity_kwh'],
            self.battery_params['soc_max'] * self.battery_params['capacity_kwh']),
    doc="State of charge energy (kWh)"
)
```

3. Power Limits

Binary-continuous variable linking to prevent simultaneous charging/discharging and enforce power rating:

$$p_{ch}(t) \leq y_{ch}(t) \cdot P_{max}^{config} \quad \forall t \in T$$
$$p_{dis}(t) \leq y_{dis}(t) \cdot P_{max}^{config} \quad \forall t \in T$$

Pyomo Implementation:

```
def charging_limit_rule(model, t):
    return model.p_ch[t] <= model.y_ch[t] * model.P_max_config

def discharging_limit_rule(model, t):
    return model.p_dis[t] <= model.y_dis[t] * model.P_max_config

model.charging_limit = pyo.Constraint(model.T, rule=charging_limit_rule)
model.discharging_limit = pyo.Constraint(model.T, rule=discharging_limit_rule)
```

4. Simultaneous Operation Prevention

Ensures battery cannot charge and discharge at the same time:

$$y_{ch}(t) + y_{dis}(t) \leq 1 \quad \forall t \in T$$

Pyomo Implementation:

```
def no_simultaneous_rule(model, t):
    return model.y_ch[t] + model.y_dis[t] <= 1

model.no_simultaneous = pyo.Constraint(model.T, rule=no_simultaneous_rule)
```

5. Market Co-optimization Power Limits

Critical Constraint: Allocates total available power between energy arbitrage and ancillary service reserves. This constraint creates the economic trade-off between DA arbitrage and AS capacity bidding:

$$p_{ch}(t) + 1000 \cdot c_{fcr}(b) + 1000 \cdot c_{afrr}^{pos}(b) \leq P_{max}^{config} \quad \forall b \in B, \forall t \in B$$
$$p_{dis}(t) + 1000 \cdot c_{fcr}(b) + 1000 \cdot c_{afrr}^{neg}(b) \leq P_{max}^{config} \quad \forall b \in B, \forall t \in B$$

Note: MW bids are converted to kW by multiplying by 1000.

Pyomo Implementation:

```
def power_ch_reserve_limit_rule(model, t):
    b = model.block_map[t] # Pre-computed block mapping
    return (model.p_ch[t] + 1000 * model.c_fcr[b] +
            1000 * model.c_afrr_pos[b] <= model.P_max_config)

def power_dis_reserve_limit_rule(model, t):
    b = model.block_map[t]
    return (model.p_dis[t] + 1000 * model.c_fcr[b] +
            1000 * model.c_afrr_neg[b] <= model.P_max_config)

model.power_ch_reserve_limit = pyo.Constraint(model.T, rule=power_ch_reserve_limit_rule)
model.power_dis_reserve_limit = pyo.Constraint(model.T, rule=power_dis_reserve_limit_rule)
```

6. Daily Cycle Limit

Limits total daily discharged energy to prevent excessive battery degradation:

$$\sum(t \in D) p_{dis}(t) \cdot \Delta t \leq N_{cycles} \cdot E_{nom} \quad \forall d \in D$$

Where N_{cycles} is the daily cycle limit (1.0, 1.5, or 2.0 cycles per day).

Pyomo Implementation:

```
def daily_cycle_rule(model, d):
    # Get time steps for this day using pre-computed mapping
    day_times = [t for t in model.T if country_data['day_id'].iloc[t] == d]
    if day_times:
        return sum(model.p_dis[t] * model.dt for t in day_times) <= \
            model.N_cycles * model.E_nom
    else:
        return pyo.Constraint.Skip

model.daily_cycle_limit = pyo.Constraint(model.D, rule=daily_cycle_rule)
```

7. Ancillary Service Energy Reserve

Critical Constraint: Maintains sufficient SOC to deliver awarded AS capacity for 15 minutes ($\Delta t=0.25$ h). This constraint **dynamically restricts the usable SOC range** based on AS commitments:

$$e_{soc}(t) \geq SOC_{min} \cdot E_{nom} + (1000 \cdot c_{fcr}(b) + 1000 \cdot c_{afrr}^{pos}(b)) \cdot \Delta t \quad \forall b \in B, \forall t \in B$$
$$e_{soc}(t) \leq SOC_{max} \cdot E_{nom} - (1000 \cdot c_{fcr}(b) + 1000 \cdot c_{afrr}^{neg}(b)) \cdot \Delta t \quad \forall b \in B, \forall t \in B$$

Important Note: When large AS bids are made, these constraints significantly reduce the effective SOC range available for cycling, which can make the daily cycle limit constraint non-binding.

Pyomo Implementation:

```
def energy_reserve_pos_rule(model, t):
    b = model.block_map[t]
    return (model.e_soc[t] >=
            model.SOC_min * model.E_nom +
            (1000 * model.c_fcr[b] + 1000 * model.c_afrr_pos[b]) * model.dt)

def energy_reserve_neg_rule(model, t):
    b = model.block_map[t]
    return (model.e_soc[t] <=
            model.SOC_max * model.E_nom -
            (1000 * model.c_fcr[b] + 1000 * model.c_afrr_neg[b]) * model.dt)

model.energy_reserve_pos = pyo.Constraint(model.T, rule=energy_reserve_pos_rule)
model.energy_reserve_neg = pyo.Constraint(model.T, rule=energy_reserve_neg_rule)
```

8. Minimum Bid Size Constraints

Enforces market rules requiring minimum bid sizes (1 MW for both FCR and aFRR) using binary-continuous variable linking:

$$c_fcr(b) \geq y_fcr(b) \cdot \text{MinBid_fcr} \quad \forall b \in B$$

$$c_fcr(b) \leq y_fcr(b) \cdot P_max^{config}/1000 \quad \forall b \in B$$

$$c_aFRR^{pos}(b) \geq y_aFRR^{pos}(b) \cdot \text{MinBid_afrr} \quad \forall b \in B$$

$$c_aFRR^{pos}(b) \leq y_aFRR^{pos}(b) \cdot P_max^{config}/1000 \quad \forall b \in B$$

$$c_aFRR^{neg}(b) \geq y_aFRR^{neg}(b) \cdot \text{MinBid_afrr} \quad \forall b \in B$$

$$c_aFRR^{neg}(b) \leq y_aFRR^{neg}(b) \cdot P_max^{config}/1000 \quad \forall b \in B$$

Pyomo Implementation:

```
# FCR constraints
def fcr_min_bid_rule(model, b):
    return model.c_fcr[b] >= model.y_fcr[b] * model.min_bid_fcr

def fcr_max_bid_rule(model, b):
    return model.c_fcr[b] <= model.y_fcr[b] * (model.P_max_config / 1000)

model.fcr_min_bid = pyo.Constraint(model.B, rule=fcr_min_bid_rule)
model.fcr_max_bid = pyo.Constraint(model.B, rule=fcr_max_bid_rule)

# Similar constraints for aFRR positive and negative
# (See model.py lines 455-472 for complete implementation)
```

9. Ancillary Service Market Mutual Exclusivity

Prevents simultaneous participation in multiple AS markets for the same block (FCR, aFRR+, aFRR- are mutually exclusive):

$$y_fcr(b) + y_aFRR^{pos}(b) + y_aFRR^{neg}(b) \leq 1 \quad \forall b \in B$$

Pyomo Implementation:

```
def as_market_exclusivity_rule(model, b):
    return model.y_fcr[b] + model.y_afrr_pos[b] + model.y_afrr_neg[b] <= 1

model.as_market_exclusivity = pyo.Constraint(model.B, rule=as_market_exclusivity_rule)
```

Model Variables and Parameters

Symbol	Definition	Unit	Type
Sets & Indices			
T	Set of 15-minute time intervals, $t \in T = \{1, \dots, 35040\}$	-	Set

Symbol	Definition	Unit	Type
B	Set of 4-hour ancillary service blocks, $b \in B$	-	Set
D	Set of 24-hour days, $d \in D$	-	Set
Parameters			
$P_{DA}(t)$	Day-ahead electricity price in interval t	EUR/MWh	Input
$P_{FCR}(b)$	FCR capacity price in block b	EUR/MW	Input
$P_{aFRR}^{pos}(b)$	Positive aFRR capacity price in block b	EUR/MW	Input
$P_{aFRR}^{neg}(b)$	Negative aFRR capacity price in block b	EUR/MW	Input
E_{nom}	Nominal energy capacity of the BESS	kWh	Input
P_{max}^{config}	Maximum charge/discharge power for configuration	kW	Input
η_{ch}, η_{dis}	Charging and discharging efficiencies	-	Input
SOC_{min}, SOC_{max}	Min/max state of charge as fraction of E_{nom}	-	Input
N_{cycles}	Daily cycle limit for the selected configuration	-	Input
e_{nom}^{init}	Initial state of charge at start	kWh	Input
Δt	Duration of a time interval (0.25)	h	Constant
Δb	Duration of an ancillary service block (4)	h	Constant
$MinBid_{fcr}$	Minimum bid size for FCR market	MW	Input
$MinBid_{afrr}$	Minimum bid size for aFRR market	MW	Input
Decision Variables			
$p_{ch}(t)$	Power used to charge the BESS in interval t	kW	Continuous ≥ 0
$p_{dis}(t)$	Power discharged from the BESS in interval t	kW	Continuous ≥ 0
$e_{soc}(t)$	Energy stored in the BESS at end of interval t	kWh	Continuous ≥ 0
$c_{FCR}(b)$	Symmetric FCR capacity bid for block b	MW	Continuous ≥ 0
$c_{aFRR}^{pos}(b)$	Positive aFRR capacity bid for block b	MW	Continuous ≥ 0
$c_{aFRR}^{neg}(b)$	Negative aFRR capacity bid for block b	MW	Continuous ≥ 0
$y_{ch}(t)$	Binary variable, 1 if charging in interval t	-	Binary
$y_{dis}(t)$	Binary variable, 1 if discharging in interval t	-	Binary
$y_{fcr}(b)$	Binary variable, 1 if bidding for FCR in block b	-	Binary
$y_{aFRR}^{pos}(b)$	Binary, 1 if bidding for positive aFRR in block b	-	Binary
$y_{aFRR}^{neg}(b)$	Binary, 1 if bidding for negative aFRR in block b	-	Binary

Investment Optimization

Objective

The primary objective of the investment optimization is to evaluate and rank the financial viability of deploying a Battery Energy Storage System (BESS) across various European countries and technical configurations. This is achieved by performing a 10-year Discounted Cash Flow (DCF) analysis to determine the Net Present Value (NPV) and the Levelized Return on Investment (ROI) for each scenario.

Financial Modeling Approach

The core of the analysis is a DCF model that correctly aligns nominal cash flows with a nominal discount rate. This approach ensures that the effects of inflation are accounted for consistently and accurately.

- **Cash Flows:** We project future profits by growing the initial year's profit (from the operational optimization) at the country-specific inflation rate. This creates a stream of **nominal cash flows**.
- **Discount Rate:** We use the country-specific Weighted Average Cost of Capital (WACC) as the **nominal discount rate**.

Step-by-Step Calculation

The analysis follows a three-step process for the results from 45-54 scenarios (5-6 countries × 9 configurations):

Step 1: Projecting Nominal Profits

For each country, we take the BEST annual profit of this country (the highest annual profit of nine different configuration settings) calculated by the Pyomo model for the year 2024, denoted as Π_{2024} , and project it over a 10-year horizon from year $y=1$ (2024) to $y=10$ (2033). The nominal profit for any future year y is calculated by applying the country's annual inflation rate, π :

$$\Pi_y = \Pi_{2024} \cdot (1 + \pi)^{(y-1)}$$

Step 2: Calculating Net Present Value (NPV)

We calculate the NPV by summing the discounted values of all future nominal profits and subtracting the initial Capital Expenditure (CAPEX). The CAPEX is given as \$200 per kWh. For a BESS with a nominal energy capacity $E_{nom} = 4.472$ MWh, the CAPEX is $200 \times 4472 = 894,400$ EUR.

The WACC is denoted by i :

$$NPV = [\sum_{y=1 \text{ to } 10} \Pi_y / (1 + i)^y] - CAPEX$$

Step 3: Calculating Levelized Return on Investment (ROI)

We calculate the Levelized ROI as specified in the competition guidelines. This metric annualizes the return relative to the initial investment:

$$\text{Levelized ROI (\%)} = [PV(\text{Total Profits}) / (CAPEX \times \text{Lifetime})] \times 100$$

Where:

- $PV(\text{Total Profits})$ is the first term in the NPV equation: $\sum_{y=1 \text{ to } 10} \Pi_y / (1 + i)^y$
- CAPEX is the initial investment cost (894,400 EUR)
- Lifetime is the project duration (10 years)

Example Calculation

Consider a hypothetical scenario:

- **BESS Configuration:** C-rate = 0.5 C, $E_{nom} = 4.472$ MWh
- **Initial CAPEX:** $200 \text{ EUR/kWh} \times 4,472 \text{ kWh} = 894,400 \text{ EUR}$
- **Country Parameters:** WACC (i) = 7.0%, Inflation (π) = 2.0%
- **Simulated Profit (2024):** $\Pi_{2024} = 50,000 \text{ EUR}$

The Levelized ROI would be approximately 9.89%, indicating the project yields an average annual return of 9.89% on its initial investment in present value terms.

Implementation Pipeline

1. Data Processing (`py_script/market_da.py`)

Purpose: Load and transform market data from Excel to optimization-ready format

The data transformation pipeline follows this structure:

```
# Data transformation pipeline
raw_data = load_market_tables("input/TechArena2025_data.xlsx")
# Input: Excel sheets (Day-ahead, FCR, aFRR prices)
# Output: Wide-format DataFrames

tidy_data = convert_to_tidy_format(raw_data)
# Input: Wide-format DataFrames
# Output: Tidy JSONL format for optimization
```

Key Functions:

- load_market_tables() : Excel → pandas DataFrames
- wide_to_tidy*() : Wide format → Tidy format conversion
- save_tidy_data() : DataFrame → JSONL export

Data Format Details:

- **Day-ahead & FCR:** columns [timestamp, DE_LU/DE, AT, CH, HU, CZ]
- **aFRR:** columns [timestamp, DE_Pos, DE_Neg, AT_Pos, AT_Neg, ...]
- **Tidy format:** [timestamp, country, price_eur_mwh/price_eur_mw]

2. Optimization Model (py_script/model.py)

Purpose: Pyomo-based mixed-integer linear programming (MILP) model for BESS optimization

Mathematical Model Structure

Objective Function: Maximize annual profit

$$\begin{aligned} \max Z = & \sum (P_{DA}(t) \times p_{dis}(t) - P_{DA}(t) \times p_{ch}(t)) \times \Delta t + \\ & \sum (P_{FCR}(b) \times c_{fcr}(b) + P_{aFRR_pos}(b) \times c_{afrr_pos}(b) + P_{aFRR_neg}(b) \times c_{afrr_neg}(b)) \times \Delta b \end{aligned}$$

Where:

- First term: Day-ahead energy arbitrage revenue
- Second term: Ancillary service capacity payments
- $P_{DA}(t)$: Day-ahead price [EUR/MWh]
- $p_{dis}(t), p_{ch}(t)$: Discharge/charge power [kW]
- $P_{FCR}(b), P_{aFRR}(b)$: Capacity prices [EUR/MW]
- $c_{fcr}(b), c_{afrr}(b)$: Capacity bids [MW]

Key Constraint Categories:

1. **SOC Dynamics:** Energy balance with efficiency losses

$$e_{soc}(t) = e_{soc}(t-1) + (p_{ch}(t) \times \eta_{ch} - p_{dis}(t) / \eta_{dis}) \times \Delta t$$

2. **Power Limits:** C-rate and binary variable constraints

$$\begin{aligned} p_{ch}(t) &\leq y_{ch}(t) \times P_{max}^{config} \\ p_{dis}(t) &\leq y_{dis}(t) \times P_{max}^{config} \\ y_{ch}(t) + y_{dis}(t) &\leq 1 \quad \# \text{ No simultaneous charge/discharge} \end{aligned}$$

3. **Market Co-optimization:** Power allocation between energy and reserves

$$\begin{aligned} p_{ch}(t) + 1000 \times c_{fcr}(b) + 1000 \times c_{afrr_pos}(b) &\leq P_{max}^{config} \\ p_{dis}(t) + 1000 \times c_{fcr}(b) + 1000 \times c_{afrr_neg}(b) &\leq P_{max}^{config} \end{aligned}$$

4. **Daily Cycle Limits:** Prevent excessive battery degradation

$$\sum (p_{dis}(t) \times \Delta t) \leq N_{cycles} \times E_{nom} \quad \forall d \in D$$

5. **Energy Reserves:** Maintain capacity for ancillary service delivery

$$e_{soc}(t) \geq SOC_{min} \times E_{nom} + (c_{fcr}(b) + c_{afrr_pos}(b)) \times 1000 \times \Delta t$$
$$e_{soc}(t) \leq SOC_{max} \times E_{nom} - (c_{fcr}(b) + c_{afrr_neg}(b)) \times 1000 \times \Delta t$$

6. **Minimum Bid Sizes:** Market participation requirements (1 MW minimum)

$$c_{fcr}(b) \geq y_{fcr}(b) \times MinBid_{fcr}$$
$$c_{afrr}(b) \geq y_{afrr}(b) \times MinBid_{afrr}$$

Pyomo Model Components

Sets and Indices:

- τ : Time intervals ($35,040 \times 15$ -min intervals for 2024)
- B : Market blocks ($2,190 \times 4$ -hour blocks for ancillary services)
- d : Days (366 days for 2024)

Decision Variables:

- $p_{ch}(t), p_{dis}(t)$: Charge/discharge power [kW]
- $e_{soc}(t)$: State of charge [kWh]
- $c_{fcr}(b)$: FCR capacity bid [MW]
- $c_{afrr_pos}(b), c_{afrr_neg}(b)$: aFRR capacity bids [MW]
- $y_{ch}(t), y_{dis}(t)$: Binary variables for operation modes
- $y_{fcr}(b), y_{afrr_pos}(b), y_{afrr_neg}(b)$: Binary variables for market participation

Model Architecture:

```
class ImprovedBESSOptimizer:
    """Main optimization class implementing MILP model"""

    def __init__(self):
        # Battery and market parameters
        # Pre-computed mappings for efficiency

    def create_model(self, country_data):
        # Define Pyomo model with sets, parameters, variables
        # Add objective function and constraints

    def solve_model(self, solver='cplex'):
        # Solve optimization with specified solver
        # Extract and validate results

    def optimize(self, country_data):
        # End-to-end optimization pipeline
        # Returns complete solution dictionary
```

3. Pyomo Framework Overview

Pyomo is a Python-based optimization modeling language that enables:

- **Algebraic Modeling:** Express optimization problems in mathematical notation
- **Solver Integration:** Interface with commercial (CPLEX, Gurobi) and open-source (HiGHS) solvers
- **Scalability:** Handle large-scale problems (70,000+ variables, 100,000+ constraints)
- **Flexibility:** Support for linear, nonlinear, integer, and stochastic programming

Key Pyomo Modules Used:

- `pyomo.environ` : Core modeling environment
- `Set()` : Define index sets for time, blocks, days
- `Param()` : Define input parameters (prices, battery specs)
- `var()` : Define decision variables with bounds and domains

- `objective()` : Define optimization objective (profit maximization)
- `constraint()` : Define constraint equations
- `SolverFactory()` : Interface with optimization solvers

Complete Pyomo Model Implementation Example:

```

import pyomo.environ as pyo

# 1. Create concrete model
model = pyo.ConcreteModel(name="BESS_Optimization")

# 2. Define sets
model.T = pyo.Set(initialize=range(35040), doc="15-min time intervals")
model.B = pyo.Set(initialize=range(2190), doc="4-hour AS blocks")
model.D = pyo.Set(initialize=range(366), doc="Days")

# 3. Define parameters - Battery specs
model.E_nom = pyo.Param(initialize=4472, doc="Battery capacity (kWh)")
model.P_max_config = pyo.Param(initialize=2236, doc="Max power (kW)")
model.eta_ch = pyo.Param(initialize=0.95, doc="Charging efficiency")
model.eta_dis = pyo.Param(initialize=0.95, doc="Discharging efficiency")
model.N_cycles = pyo.Param(initialize=1.0, doc="Daily cycle limit")
model.dt = pyo.Param(initialize=0.25, doc="Time step (h)")
model.db = pyo.Param(initialize=4.0, doc="Block duration (h)")

# Market prices (dictionaries indexed by time or block)
model.P_DA = pyo.Param(model.T, initialize=da_price_dict, doc="DA price (EUR/MWh)")
model.P_FCR = pyo.Param(model.B, initialize=fcr_price_dict, doc="FCR price (EUR/MW/h)")
model.P_afrr_pos = pyo.Param(model.B, initialize=afrr_pos_dict, doc="aFRR+ price")
model.P_afrr_neg = pyo.Param(model.B, initialize=afrr_neg_dict, doc="aFRR- price")

# Block mapping parameter (pre-computed for efficiency)
model.block_map = pyo.Param(model.T, initialize=time_to_block_dict)

# 4. Define decision variables - Continuous
model.p_ch = pyo.Var(model.T, bounds=(0, 2236), doc="Charging power (kW)")
model.p_dis = pyo.Var(model.T, bounds=(0, 2236), doc="Discharging power (kW)")
model.e_soc = pyo.Var(model.T, bounds=(0, 4472), doc="SOC (kWh)")
model.c_fcr = pyo.Var(model.B, bounds=(0, 2.236), doc="FCR bid (MW)")
model.c_afrr_pos = pyo.Var(model.B, bounds=(0, 2.236), doc="aFRR+ bid (MW)")
model.c_afrr_neg = pyo.Var(model.B, bounds=(0, 2.236), doc="aFRR- bid (MW)")

# Binary variables
model.y_ch = pyo.Var(model.T, domain=pyo.Binary, doc="Charging state")
model.y_dis = pyo.Var(model.T, domain=pyo.Binary, doc="Discharging state")
model.y_fcr = pyo.Var(model.B, domain=pyo.Binary, doc="FCR participation")
model.y_afrr_pos = pyo.Var(model.B, domain=pyo.Binary, doc="aFRR+ participation")
model.y_afrr_neg = pyo.Var(model.B, domain=pyo.Binary, doc="aFRR- participation")

# 5. Define objective function
def objective_rule(model):
    # Day-ahead arbitrage profit (kW to MW conversion: /1000)
    da_profit = sum(
        (model.P_DA[t]/1000) * model.p_dis[t] * model.dt -
        (model.P_DA[t]/1000) * model.p_ch[t] * model.dt
        for t in model.T
    )
    # Ancillary service capacity revenue
    as_revenue = sum(
        model.P_FCR[b] * model.c_fcr[b] * model.db +
        model.P_afrr_pos[b] * model.c_afrr_pos[b] * model.db +
        model.P_afrr_neg[b] * model.c_afrr_neg[b] * model.db
        for b in model.B
    )
    return da_profit + as_revenue

model.objective = pyo.Objective(rule=objective_rule, sense=pyo.maximize)

# 6. Define constraints (9 types total)

# Constraint 1: SOC Dynamics

```

```

def soc_dynamics_rule(model, t):
    if t == 0:
        return model.e_soc[t] == 2236 + \
            (model.eta_ch * model.p_ch[t] - model.p_dis[t]/model.eta_dis) * model.dt
    else:
        return model.e_soc[t] == model.e_soc[t-1] + \
            (model.eta_ch * model.p_ch[t] - model.p_dis[t]/model.eta_dis) * model.dt

model.soc_dynamics = pyo.Constraint(model.T, rule=soc_dynamics_rule)

# Constraint 2: Power Limits (via variable bounds - already defined above)

# Constraint 3: Binary-continuous linking
def charging_limit_rule(model, t):
    return model.p_ch[t] <= model.y_ch[t] * model.P_max_config

def discharging_limit_rule(model, t):
    return model.p_dis[t] <= model.y_dis[t] * model.P_max_config

model.charging_limit = pyo.Constraint(model.T, rule=charging_limit_rule)
model.discharging_limit = pyo.Constraint(model.T, rule=discharging_limit_rule)

# Constraint 4: No simultaneous charge/discharge
def no_simultaneous_rule(model, t):
    return model.y_ch[t] + model.y_dis[t] <= 1

model.no_simultaneous = pyo.Constraint(model.T, rule=no_simultaneous_rule)

# Constraint 5: Market co-optimization power limits
def power_ch_reserve_rule(model, t):
    b = model.block_map[t]
    return (model.p_ch[t] + 1000*model.c_fcr[b] +
            1000*model.c_afrr_pos[b] <= model.P_max_config)

def power_dis_reserve_rule(model, t):
    b = model.block_map[t]
    return (model.p_dis[t] + 1000*model.c_fcr[b] +
            1000*model.c_afrr_neg[b] <= model.P_max_config)

model.power_ch_reserve = pyo.Constraint(model.T, rule=power_ch_reserve_rule)
model.power_dis_reserve = pyo.Constraint(model.T, rule=power_dis_reserve_rule)

# Constraint 6: Daily cycle limit
def daily_cycle_rule(model, d):
    day_times = [t for t in model.T if t//96 == d] # 96 intervals per day
    if day_times:
        return sum(model.p_dis[t] * model.dt for t in day_times) <= \
            model.N_cycles * model.E_nom
    return pyo.Constraint.Skip

model.daily_cycle_limit = pyo.Constraint(model.D, rule=daily_cycle_rule)

# Constraint 7: Energy reserves for AS delivery
def energy_reserve_pos_rule(model, t):
    b = model.block_map[t]
    return model.e_soc[t] >= (1000*model.c_fcr[b] + 1000*model.c_afrr_pos[b]) * model.dt

def energy_reserve_neg_rule(model, t):
    b = model.block_map[t]
    return model.e_soc[t] <= model.E_nom - \
        (1000*model.c_fcr[b] + 1000*model.c_afrr_neg[b]) * model.dt

model.energy_reserve_pos = pyo.Constraint(model.T, rule=energy_reserve_pos_rule)
model.energy_reserve_neg = pyo.Constraint(model.T, rule=energy_reserve_neg_rule)

```

```

# Constraint 8: Minimum bid sizes
def fcr_min_bid_rule(model, b):
    return model.c_fcr[b] >= model.y_fcr[b] * 1.0 # 1 MW minimum

def fcr_max_bid_rule(model, b):
    return model.c_fcr[b] <= model.y_fcr[b] * (model.P_max_config/1000)

model.fcr_min_bid = pyo.Constraint(model.B, rule=fcr_min_bid_rule)
model.fcr_max_bid = pyo.Constraint(model.B, rule=fcr_max_bid_rule)
# Similar for aFRR+ and aFRR-

# Constraint 9: AS market mutual exclusivity
def as_exclusivity_rule(model, b):
    return model.y_fcr[b] + model.y_afrr_pos[b] + model.y_afrr_neg[b] <= 1

model.as_exclusivity = pyo.Constraint(model.B, rule=as_exclusivity_rule)

# 7. Solve the model
solver = pyo.SolverFactory('cplex')
solver.options['mipgap'] = 0.01 # 1% optimality gap
solver.options['timelimit'] = 600 # 10 minutes
results = solver.solve(model, tee=True)

# 8. Extract and analyze results
if results.solver.status == pyo.SolverStatus.ok:
    total_profit = pyo.value(model.objective)
    soc_schedule = [pyo.value(model.e_soc[t]) for t in model.T]
    power_schedule = [(pyo.value(model.p_ch[t]), pyo.value(model.p_dis[t]))
                      for t in model.T]
    fcr_bids = [pyo.value(model.c_fcr[b]) for b in model.B]

    print(f"Total Annual Profit: €{total_profit:,.2f}")
    print(f"Average SOC: {sum(soc_schedule)/len(soc_schedule):.1f} kWh")

```

Model Statistics:

- **Variables:** ~105,000 total (70,000 continuous + 35,000 binary)
- **Constraints:** ~140,000 total (35K SOC dynamics + 105K market/power constraints)
- **Solve Time:** 3-10 minutes per scenario with CPLEX (600s time limit)
- **Memory Usage:** ~2-3 GB per optimization instance

4. Usage Examples

Basic Single Scenario Optimization

```
from py_script.model import ImprovedBESSOptimizer

# Initialize optimizer
optimizer = ImprovedBESSOptimizer()

# Load and preprocess data
data_file = "data/TechArena2025_data_tidy.jsonl"
market_data = optimizer.load_and_preprocess_data(data_file)

# Extract country-specific data
country_data = optimizer.extract_country_data(market_data, country='DE')

# Run optimization for specific configuration
results = optimizer.optimize(
    country_data=country_data,
    c_rate=0.5,          # 0.5C = 2236 kW max power
    daily_cycle_limit=2.0 # 2 full cycles per day
)

# Access results
print(f"Total Profit: €{results['total_profit']:,.2f}")
print(f"DA Revenue: €{results['da_revenue']:,.2f}")
print(f"AS Revenue: €{results['as_revenue']:,.2f}")
print(f"Solve Time: {results['solve_time']:0.1f}s")
print(f"Solver Status: {results['solver_status']}")
```

Comprehensive Scenario Analysis

```
# Run all country/configuration scenarios (54 total)
results_df = optimizer.run_scenario_analysis(
    data_file="data/TechArena2025_data_tidy.jsonl",
    output_file="results/scenario_analysis.csv",
    num_days=365 # Full year simulation
)

# Display top 10 configurations by profit
print(results_df.sort_values('total_profit', ascending=False).head(10))

# Best configuration per country
best_configs = results_df.loc[results_df.groupby('country')['total_profit'].idxmax()]
print("\nBest Configuration per Country:")
print(best_configs[['country', 'c_rate', 'daily_cycle_limit', 'total_profit']])

# Revenue breakdown analysis
print("\nRevenue Sources:")
print(results_df.groupby('country')[['da_revenue', 'as_revenue']].mean())
```

Visualization of Results

```
from py_script.market_da import (
    plot_battery_operation_schedule,
    plot_market_price_bid_comparison,
    plot_revenue_breakdown
)

# Plot battery operation for a week
plot_battery_operation_schedule(
    soc_data=results['soc_schedule'],
    power_data=results['power_schedule'],
    time_range=(0, 672), # First week (7 days x 96 intervals)
    save_path='plots/battery_operation_week1.html'
)

# Compare market prices and actual bids
plot_market_price_bid_comparison(
    da_prices=market_data['price_day_ahead'],
    fcr_prices=market_data['price_fcr'],
    fcr_bids=results['fcr_bids'],
    save_path='plots/market_comparison.html'
)

# Revenue breakdown pie chart
plot_revenue_breakdown(
    da_revenue=results['da_revenue'],
    fcr_revenue=results['fcr_revenue'],
    afrr_revenue=results['afrr_revenue'],
    save_path='plots/revenue_breakdown.html'
)
```

5. Investment Analysis Framework

10-Year DCF Analysis:

- **CAPEX:** €894,400 (4,472 kWh × €200/kWh)
- **Country-specific WACC and inflation rates**
- **Net Present Value (NPV) calculation**
- **Levelized ROI over 10-year period**

Financial Model:

```
# DCF calculation
for year in range(1, 11):
    inflated_profit = base_profit * (1 + inflation)^(year-1)
    pv_profit = inflated_profit / (1 + discount_rate)^year
    cumulative_npv += pv_profit

levelized_roi = (cumulative_npv / capex) / 10 * 100
```

Implementation Example:

```

from py_script.investment_analysis import InvestmentAnalyzer

# Run investment analysis with DCF model
analyzer = InvestmentAnalyzer()
investment_results = analyzer.analyze_investments(
    operational_results=results_df, # From scenario analysis
    capex_per_kwh=200, # EUR/kWh
    lifetime_years=10
)

# View NPV and ROI rankings
print(investment_results.sort_values('levelized_roi', ascending=False))

# Country-wise investment summary
print("\nInvestment Metrics by Country:")
for country in ['DE', 'AT', 'CH', 'HU', 'CZ']:
    country_data = investment_results[investment_results['country'] == country]
    best = country_data.loc[country_data['levelized_roi'].idxmax()]
    print(f"{country}: NPV=€{best['npv']:,.0f}, ROI={best['levelized_roi']:.2f}%")

```

6. Performance Metrics and Expected Results

Computational Performance (based on full year validation with CPLEX):

- **Problem Size:**
 - 35,040 time intervals (15-min resolution for 2024)
 - ~105,000 decision variables (70K continuous + 35K binary)
 - ~140,000 constraints
- **Runtime:** 3-10 minutes per scenario (600s time limit)
- **Total Execution:** ~5-8 hours for all 54 scenarios (6 countries × 9 configurations)
- **Memory Usage:** 2-4 GB peak per scenario
- **Success Rate:** 100% optimal solutions (with 1% MIP gap tolerance)

Optimization Results Summary:

- **Annual Revenue Range:** €300,000 - €800,000 (varies by country and configuration)
- **Revenue Split:**
 - Day-ahead arbitrage: 20-40% of total revenue
 - Ancillary services (FCR/aFRR): 60-80% of total revenue
- **Best Configurations:**
 - High C-rate (0.5C) typically performs best for power-constrained scenarios
 - Higher cycle limits (2.0) allow more flexibility but may not be binding when AS dominates
- **Country Rankings** (by profit potential):
 - Germany (DE/DE_LU): Highest FCR prices
 - Switzerland (CH): Strong AS markets
 - Austria (AT): Balanced DA/AS opportunities
 - Czech Republic (CZ): Moderate performance
 - Hungary (HU): Lower AS prices

Investment Analysis Results:

- **CAPEX:** €894,400 (4,472 kWh × €200/kWh)
- **NPV Range:** €1.5M - €4.5M (10-year horizon, varies by country)
- **Levelized ROI:** 8% - 15% annually (after accounting for WACC and inflation)
- **Payback Period:** 3-6 years in most scenarios

Key Insights:

1. **AS Revenue Dominance:** Ancillary services provide 60-80% of total revenue, making them the primary value driver
2. **Energy Reserve Bottleneck:** Large AS commitments restrict usable SOC range, often making daily cycle limits non-binding
3. **Market Co-optimization:** Power allocation between DA arbitrage and AS capacity is the critical trade-off

4. **Country Selection Impact:** Location choice has 2-3× impact on profitability due to market price differences

Installation and Setup

Prerequisites

- Python 3.9+
- Optimization solver (CPLEX, Gurobi, or HiGHS)
- Required Python packages (see requirements.txt)

Installation

1. Install Dependencies:

```
pip install -r requirements.txt
```

2. Verify Solver Installation:

```
python -c "import pyomo.environ as pyo; print(pyomo.SolverFactory('cplex').available())"
```

Execution

1. Run Full Analysis:

```
python final_45_scenarios.py
```

2. Generate Submission Files:

```
python submission_generator.py
```

3. Verify Results:

```
ls output/ # Should contain 3 CSV files
```

Quick Test

For quick validation, run a subset of scenarios:

```
python -c "  
from final_45_scenarios import run_scenario  
result = run_scenario('AT', 0.5, 2.0)  
print(f'Revenue: €{result[\"total_revenue\"]:,.0f}')  
"
```

Output Files

Required TechArena 2025 Submissions

1. **TechArena_Phase1_Configuration.csv** : Configuration analysis results
 - Columns: C-rate, number of cycles, yearly profits [KEUR/MW], levelized ROI [%]
2. **TechArena_Phase1_Investment.csv** : 10-year DCF investment analysis
 - Columns: WACC, inflation rate, discount rate, yearly profits, PV yearly profits, Cumulative NPV, Levelized ROI [%]
3. **TechArena_Phase1_Operation.csv** : Optimal operation schedule (best scenario)
 - Columns: Timestamp, Stored energy [MWh], SoC [-], Charge [MWh], Discharge [MWh], FCR bid [MW], aFRR pos/neg bid [MW]

Technical Validation

-  All 45 scenarios tested and validated

-  Solution feasibility verified
-  Market constraint compliance confirmed
-  Revenue calculations validated
-  DCF analysis implemented
-  CSV output formats comply with TechArena requirements

Project Structure

```
TechArena2025_EMS/
├── py_script/
│   ├── market_da.py           # Data processing and loading
│   ├── model.py               # Pyomo optimization model
│   └── submission_generator.py # CSV output generation
├── data/
│   ├── TechArena2025_data.xlsx
│   └── TechArena2025_data_tidy.jsonl
├── jb_notebook/
│   ├── final_test.md          # Complete implementation plan
│   └── Test_script.ipynb      # Validation notebooks
├── SoloGen_TechArena2025_Phase1_submission/
│   ├── main.py                # Main execution script
│   ├── README.md              # This file
│   ├── requirements.txt        # Dependencies
│   ├── input/
│   │   └── TechArena2025_data.xlsx
│   └── output/                 # Generated CSV files
│       ├── TechArena_Phase1_Configuration.csv
│       ├── TechArena_Phase1_Investment.csv
│       └── TechArena_Phase1_Operation.csv
├── requirements.txt
└── README.md
```

Team Information

Team Name: SoloGen
Challenge: Huawei TechArena 2025 Phase I
Implementation: Python 3.9+ with Pyomo optimization framework
Key Technologies: Pyomo, CPLEX/Gurobi/HiGHS, pandas, numpy
Contact: [Contact Information]

Model Insights and Technical Notes

Critical Constraint Interactions

Energy Reserve Constraints vs. Daily Cycle Limits

Key Finding: The energy reserve constraints (Constraint 7) often dominate the daily cycle limit constraints (Constraint 6), making cycle limits non-binding in many scenarios.

Mechanism:

1. When the optimizer awards large AS capacity bids (e.g., 2 MW FCR), the energy reserve constraints require maintaining sufficient SOC buffers:

$$\text{Lower Reserve} = (1000 \times c_{fcr} + 1000 \times c_{afrr_pos}) \times 0.25h = AS_MW \times 250 \text{ kWh}$$
$$\text{Upper Reserve} = (1000 \times c_{fcr} + 1000 \times c_{afrr_neg}) \times 0.25h = AS_MW \times 250 \text{ kWh}$$

2. These reserves **shrink the usable SOC range:**

```
Effective Capacity = E_nom - Lower_Reserve - Upper_Reserve
                   = 4472 - 500 - 500 = 3472 kWh (for 2 MW AS bid)
```

3. Daily cycle limit allows:

```
Max Daily Discharge = N_cycles × E_nom = 2.0 × 4472 = 8944 kWh
```

4. But physical constraint from reserves:

```
Actual Max Discharge = Effective_Capacity = 3472 kWh << 8944 kWh
```

Implication: Higher cycle limits (1.5, 2.0) provide little additional value when AS markets dominate, because the optimizer is constrained by energy reserves, not cycle limits.

Market Co-optimization Trade-offs

Power Allocation Trade-off (Constraint 5):

```
p_ch(t) + 1000·c_fcr(b) + 1000·c_aFRR_pos(b) ≤ P_max_config
p_dis(t) + 1000·c_fcr(b) + 1000·c_aFRR_neg(b) ≤ P_max_config
```

This creates a **zero-sum game** between:

- **Day-ahead arbitrage:** Requires power to charge/discharge → uncertain revenue (price spreads vary)
- **Ancillary services:** Require power held in reserve → guaranteed revenue (capacity payments)

Economic Behavior:

- AS markets offer **stable, guaranteed revenue** for capacity reservation
- DA arbitrage offers **volatile, opportunity-based revenue** from price spreads
- When AS prices are high (typical in Germany, Switzerland), optimizer prefers AS commitments
- This allocates most power to reserves, leaving minimal capacity for DA arbitrage

Result: AS revenue typically contributes 60-80% of total profit across most scenarios.

Solver Performance Considerations

Problem Characteristics:

- **Type:** Mixed Integer Linear Programming (MILP)
- **Scale:** Large (105K variables, 140K constraints)
- **Sparsity:** High (most constraints involve only local time/block indices)
- **Difficulty:** Medium (binary variables for operational states and market participation)

Solver Configuration:

```
solver.options['mipgap'] = 0.01      # 1% optimality gap tolerance
solver.options['timelimit'] = 600    # 10-minute time limit per scenario
solver.options['threads'] = 4        # Parallel processing threads
```

Performance Tips:

1. **Pre-compute mappings:** Block-to-time mappings avoid expensive lookups in constraints
2. **Index prices appropriately:** AS prices by block (not time) reduces memory by 16×
3. **Warm start:** Use previous solution as initial point for similar scenarios
4. **Parallel scenarios:** Run multiple countries/configurations simultaneously on multi-core systems

Data Processing Pipeline

Timestamp Alignment:

- Critical issue: Raw data timestamps may not align to exact 15-minute boundaries

- Solution: Round all timestamps to nearest 15-minute interval before processing

```
df['timestamp'] = pd.to_datetime(df['timestamp']).dt.round('15min')
```

Block Structure Validation:

- Each 4-hour AS block must contain exactly 16 time intervals ($4h \div 0.25h$)
- Gaps or misalignments cause constraint indexing errors
- Validation included in `_validate_input_data()` method

Model Validation Results

Constraint Verification (against LaTeX mathematical formulation):

- ☒ SOC dynamics correctly implement efficiency losses
- ☒ Power limits properly linked with binary variables
- ☒ Market co-optimization enforces power allocation trade-offs
- ☒ Daily cycle limits correctly sum discharge energy per day
- ☒ Energy reserves maintain AS delivery capability
- ☒ Minimum bid sizes enforce 1 MW market requirements
- ☒ AS market exclusivity prevents simultaneous bidding

Bug Fix History:

- **Issue:** Daily cycle limit was incorrectly multiplied by `num_days` in earlier versions
- **Impact:** Made constraint 7× looser for weekly, 365× looser for annual scenarios
- **Resolution:** Removed `num_days` multiplication; constraint now operates per-day as specified
- **Verification:** Optimal solutions remained unchanged after fix (confirming AS reserves were the binding constraint)

Key Concepts

Battery Energy Storage System (BESS)

At its core, a Battery Energy Storage System (BESS) is a large, rechargeable battery. Its main job on the power grid is to absorb electrical energy, store it, and then release it when it's most needed or most profitable.

Important: A BESS cannot simultaneously charge and discharge. At any given moment, the net flow of power is either into the battery (charging) or out of the battery (discharging).

Energy Management System (EMS)

If the BESS is the muscle, the Energy Management System (EMS) is the brain that tells the BESS what to do and when. It constantly analyzes data, such as:

- Market Electricity Prices
- Battery's State of Charge (SoC)
- Grid Frequency
- Physical Limits of the Battery (like C-rate)

C-Rate

The C-rate is a measure of how quickly a battery can be charged or discharged relative to its total capacity. A 1C rate means the battery can be fully charged or discharged in one hour. A 0.5C rate means it would take two hours.

Example: For a 4,472 kWh battery:

- 0.25C = 1,118 kW (takes 4 hours to fully charge)
- 0.50C = 2,236 kW (takes 2 hours to fully charge)

Daily Cycle Limit

This is a warranty constraint that limits how much you can use the battery each day to prevent it from degrading too quickly. One "cycle" is equivalent to one full charge and one full discharge.

Example:

- 1.0 cycles/day: Can discharge 4,472 kWh per day (one full capacity)
- 2.0 cycles/day: Can discharge 8,944 kWh per day (twice the capacity)

Market Terminology

Day-Ahead Wholesale Market: A blind auction that closes at 12:00 CET on the day before delivery (Day D-1). It determines the price for all 15-minute intervals of the following day (Day D).

Frequency Containment Reserve (FCR): The ultra-fast, automatic response to tiny deviations in grid frequency. FCR is like the **cruise control** in a car. It makes constant, tiny, automatic adjustments to the engine to keep your speed perfectly steady. Bids must be symmetric (equal capacity for both positive and negative reserves). Minimum bid: 1 MW.

Automatic Frequency Restoration Reserve (aFRR): The second line of defense for larger frequency deviations. aFRR is you, the driver, noticing you're slowing down on a hill and gently pressing the accelerator to get back to your desired speed. Unlike FCR, aFRR bids are asymmetric (different amounts for positive and negative). Minimum bid: 1 MW.

Investment Terminology

Return on Investment (ROI): The ultimate scorecard for the project. It's a simple ratio that tells you how much profit you made relative to the initial cost.

$$\text{ROI} = (\text{Total Profit over 10 years} / \text{Initial Investment Cost}) \times 100\%$$

Weighted-Average Cost of Capital (WACC): The average interest rate a company has to pay to borrow the money needed to build the BESS. Think of it as the project's "cost of money." A higher WACC will eat into your profits and therefore lower your ROI.

Inflation Rate: The rate at which money loses its value over time. EUR 100 in your pocket today will buy less in a year. When calculating 10-year revenue, you must account for inflation. The profits you make in Year 9 are worth less than the profits you make in Year 1.

Pyomo Best Practices (Implemented)

1. Use Parameters Instead of External Data in Constraints

✓ **Good:** Use model parameters

```
def constraint_rule(model, t):
    b = model.block_map[t] # Uses model parameter
    return model.p_ch[t] + 1000 * model.c_fcr[b] <= model.P_max_config
```

✗ **Bad:** Access external data (closure anti-pattern)

```
def constraint_rule(model, t):
    b = country_data['block_id'].iloc[t] # Closure over external data
    return model.p_ch[t] + 1000 * model.c_fcr[b] <= model.P_max_config
```

2. Pre-compute Mappings for Performance

Build dictionaries before model construction and store as model parameters:

```

# Pre-compute block-to-time mapping
block_to_times = {}
for t in T_data:
    block_id = int(country_data['block_id'].iloc[t])
    if block_id not in block_to_times:
        block_to_times[block_id] = []
    block_to_times[block_id].append(t)

# Store as model parameter
model.block_map = pyo.Param(model.T, initialize=time_to_block)

```

3. Index Prices Appropriately

- DA prices vary every 15 minutes → index by time `model.T`
- AS prices constant for 4 hours → index by block `model.B`

This reduces memory usage and improves performance.

4. Efficient Objective Function

Avoid nested loops. Use appropriate indexing:

```

# Efficient: O(T) + O(B)
def objective_rule(model):
    da_revenue = sum(model.p_dis[t] * model.P_DA[t] for t in model.T)
    as_revenue = sum(model.c_fcr[b] * model.P_FCR[b] for b in model.B)
    return da_revenue + as_revenue

# Inefficient: O(B×T)
def objective_rule(model):
    return sum(
        model.c_fcr[b] * model.P_FCR[b]
        for b in model.B
        for t in model.T if model.block_map[t] == b
    )

```

5. Comprehensive Input Validation

Always validate data before model construction:

- Check for missing values
- Validate data types and ranges
- Check for null values
- Validate temporal consistency

License

This project is developed for the Huawei TechArena 2025 challenge and follows the competition guidelines and terms.

Author

Gen's BESS Optimization Team
September 2025